In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Flatten, Dense
```

In [2]:
```
train_data = pd.read_csv("sign_mnist_train.csv")
test_data = pd.read_csv("sign_mnist_test.csv")
```

In [4]:
```
# Separate labels and pixel values
y_train = train_data.iloc[:, 0].values
X_train = train_data.iloc[:, 1:].values

y_test = test_data.iloc[:, 0].values
X_test = test_data.iloc[:, 1:].values
```

In [5]:
```
# Normalize pixel values from 0-255 to 0-1
X_train = X_train / 255.0
X_test = X_test / 255.0

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (27455, 784)
Testing data: (7172, 784)
```

In [7]:
```
# c. Define the FNN architecture
model = Sequential([
    Flatten(input_shape=(784,)),
    Dense(128, activation="relu"),
    Dense(25, activation="softmax")
])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_2 (Dense)	(None, 128)	100,480
dense_3 (Dense)	(None, 25)	3,225

Total params: 103,705 (405.10 KB)

Trainable params: 103,705 (405.10 KB)

Non-trainable params: 0 (0.00 B)

In [8]:
```
# d. Compile and train the model using SGD
model.compile(
    optimizer="sgd",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1
)
```

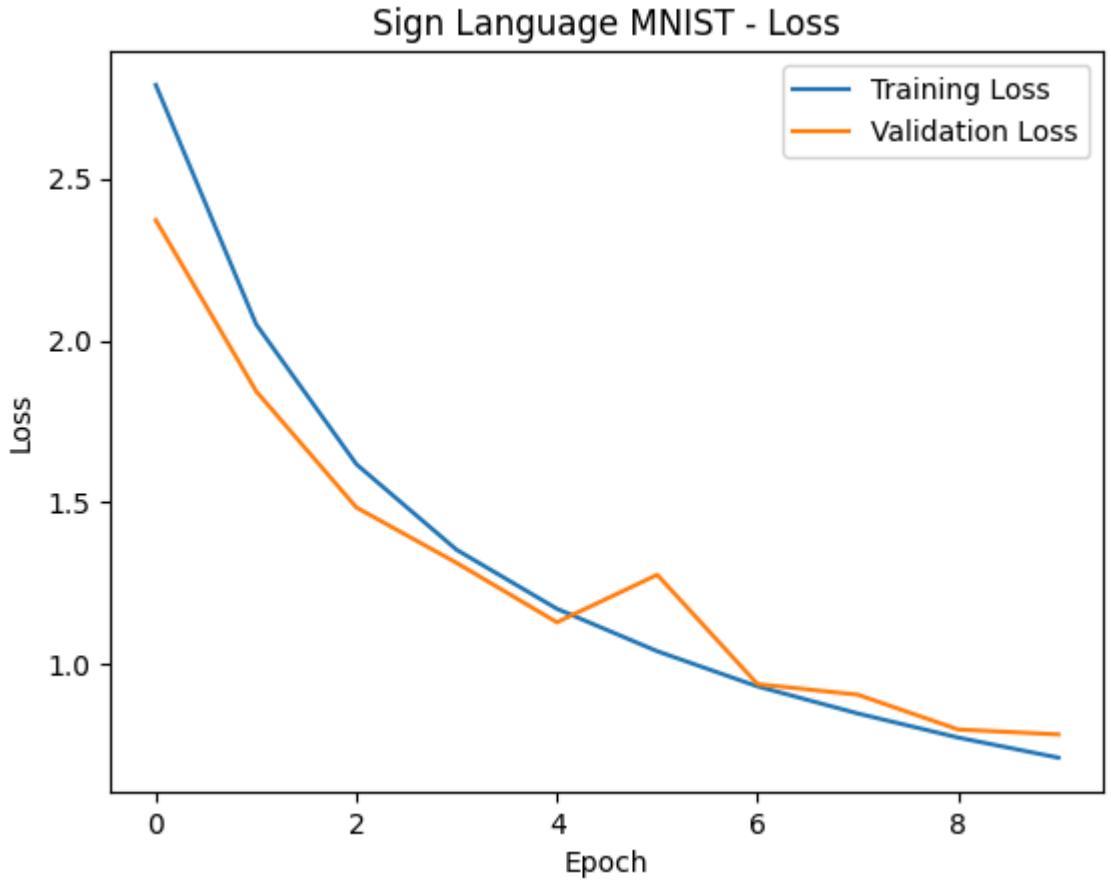
```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.2278 - loss: 2.7881 - val_accuracy: 0.3722 - val_loss: 2.3713
Epoch 2/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.4578 - loss: 2.0501 - val_accuracy: 0.4581 - val_loss: 1.8435
Epoch 3/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.5636 - loss: 1.6180 - val_accuracy: 0.5798 - val_loss: 1.4847
Epoch 4/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.6357 - loss: 1.3539 - val_accuracy: 0.5670 - val_loss: 1.3135
Epoch 5/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.6851 - loss: 1.1720 - val_accuracy: 0.6966 - val_loss: 1.1303
Epoch 6/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7182 - loss: 1.0411 - val_accuracy: 0.5761 - val_loss: 1.2769
Epoch 7/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7484 - loss: 0.9325 - val_accuracy: 0.7210 - val_loss: 0.9395
Epoch 8/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7720 - loss: 0.8490 - val_accuracy: 0.7167 - val_loss: 0.9070
Epoch 9/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.7958 - loss: 0.7747 - val_accuracy: 0.7749 - val_loss: 0.7996
Epoch 10/10
773/773 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8150 - loss: 0.7126 - val_accuracy: 0.7527 - val_loss: 0.7843
```

In [9]:
```python
# e. Evaluate the network
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)

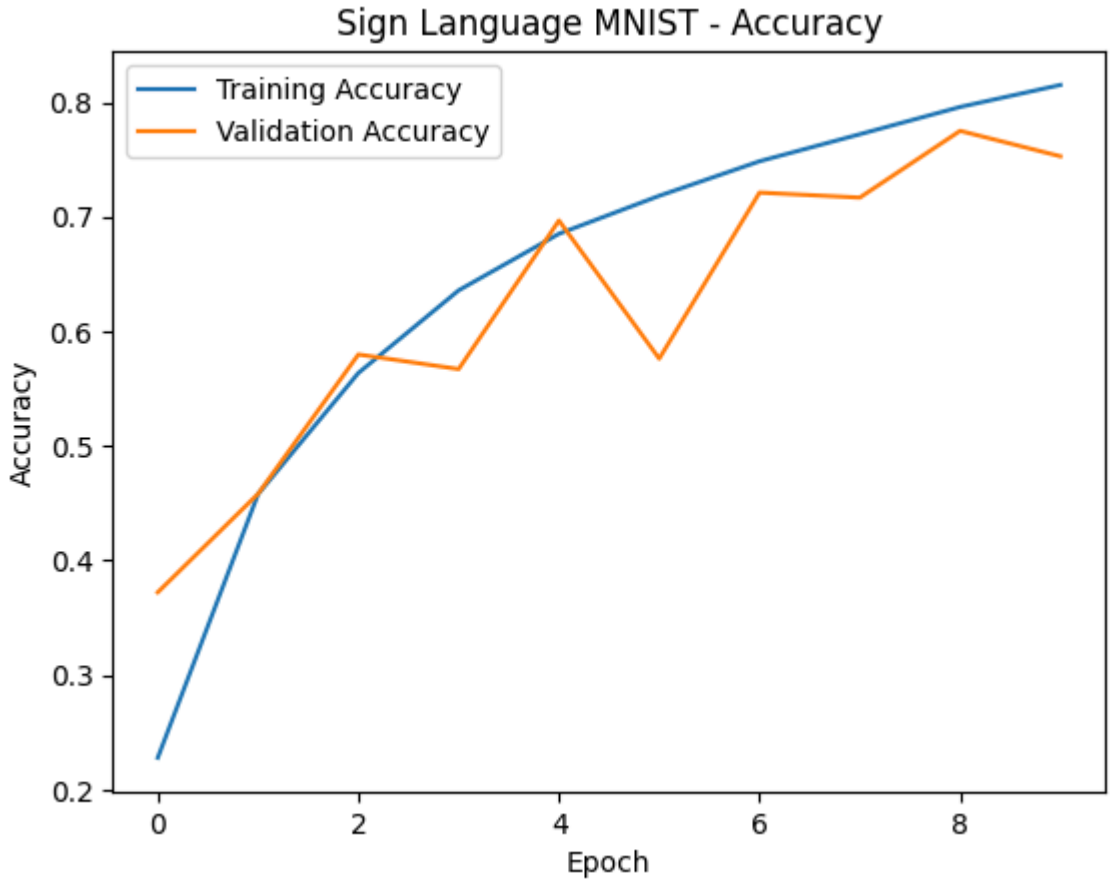
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

```
Test Loss: 1.18425452709198
Test Accuracy: 0.6226993799209595
```

In [10]:
```python
# f. Plot training and validation loss
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Sign Language MNIST - Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



In [11]:
```
# Plot training and validation accuracy
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Sign Language MNIST - Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```


Sign Language MNIST - Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0
2
4
6
8

In [ ]: